

Core Logic Implementation, Unit Testing, and Version Control Management

AI Governance Risk and Approval Platform for Nonprofit Organizations

Christopher Tucker

MSIT 5910 Capstone Project

University of the People

September 24, 2026

Part A: Core Logic Implementation and Output Validation

Alignment with Project Objectives

The primary objective of my capstone is to provide nonprofit organizations with a simple, consistent, and auditable way to identify and govern artificial intelligence use. Unit 4 demonstrated the end-to-end workflow in a browser. For this unit, I separated the most important business rules into testable modules: an explainable advisory risk engine and a human decision workflow. This directly supports the project's main objectives of consistent risk classification, transparency, human accountability, and traceability. The system is intentionally designed so that the algorithm informs review but never becomes the final governance authority.

The risk engine evaluates five MVP dimensions: data sensitivity, impact on clients or employees, AI autonomy, limited human oversight, and limited vendor assurance. Each input is rated from 0 through 3. The initial equal-weight score maps 0-4 to Low, 5-8 to Moderate, 9-12 to High, and 13-15 to Restricted. The implementation also applies mandatory escalation rules because a simple total can hide important combinations. For example, high autonomy combined with high individual impact forces Restricted review, while sensitive data combined with limited vendor assurance requires at least High review. The returned object preserves the score, tier, factor contributions, explanations, escalation rules, and algorithm version. This makes the result understandable rather than opaque.

```

1 // Advisory scoring and mandatory escalation
2 if (!Number.isInteger(value) || value < 0 || value > 3) {
3   throw new RangeError(`${name} must be an integer from 0 through 3.`);
4 }
5 }
6
7 function maxTier(currentTier, minimumTier) {
8   return TIER_ORDER.indexOf(currentTier) >= TIER_ORDER.indexOf(minimumTier)
9     ? currentTier
10    : minimumTier;
11 }
12
13 /**
14  * Calculates an explainable advisory risk tier.
15  *
16  * The algorithm never makes an approval decision. It only returns a preliminary
17  * tier used to guide human review and routing.
18  */
19 export function calculateRiskAssessment(input, config = DEFAULT_RISK_CONFIG) {
20   const normalized = {};
21   const factorResults = [];
22
23   for (const [name, definition] of Object.entries(config.factors)) {
24     const value = Number(input[name]);
25     assertRiskValue(name, value);
26     normalized[name] = value;
27
28     factorResults.push({
29       key: name,
30       label: definition.label,
31       value,
32       weight: definition.weight,
33       contribution: value * definition.weight
34     });
35   }
36
37   const score = factorResults.reduce(
38     (total, factor) => total + factor.contribution,
39     0
40   );
41
42   let tier = "Low";
43   if (score >= config.thresholds.restricted) tier = "Restricted";
44   else if (score >= config.thresholds.high) tier = "High";
45   else if (score >= config.thresholds.moderate) tier = "Moderate";
46
47   const contributingFactors = factorResults
48     .filter((factor) => factor.value >= 3)
49     .map((factor) => config.factors[factor.key].highExplanation);
50
51   const escalationRules = [];
52
53   if (normalized.autonomy >= 3 && normalized.affectedPeople >= 3) {
54     tier = "Restricted";
55     escalationRules.push(
56       "High-impact use combined with high AI autonomy requires restricted-level review."
57     );
58   }
59
60   if (normalized.dataSensitivity >= 3 && normalized.vendorAssurance >= 3) {
61     tier = maxTier(tier, "High");
62     escalationRules.push(
63       "Sensitive data combined with limited vendor assurance requires high-risk review."
64     );
65   }

```

Figure 1. Essential risk-engine code showing score thresholds and mandatory escalation rules.

Output validation used a synthetic AI-assisted documentation scenario. The sample produced a score of 10 and a High advisory tier because sensitive information and high impact were present. The output also retained the contributing factors and explicitly marked the result as advisory only. The same demonstration then recorded a Conditional human decision with a required rationale and generated a timestamped audit event. These outputs confirm that the implementation separates algorithmic assessment from human authority while preserving evidence for later review.

A terminal window with a dark background and light-colored text. The title bar at the top reads "npm run demo:logic - validated core output". The output is a JSON object with three main sections: "Advisory Risk Assessment", "Human Decision", and "Audit Event".

```
npm run demo:logic - validated core output

=== Advisory Risk Assessment ===
{
  "score": 10,
  "tier": "High",
  "Sensitive or regulated data",
  "High impact on clients or employees"
  "advisoryOnly": true
}
=== Human Decision ===
{
  "decision": "Conditional",
  "rationale": "Human verification is required before AI-generated content enters the official record.",
}
=== Audit Event ===
{
  "action": "DECISION_RECORDED",
  "newStatus": "CONDITIONALLY_APPROVED",
  "timestamp": "2026-09-24T18:00:05.000Z"
}
```

Figure 2. Core logic output showing the advisory risk result, human decision, and audit event.

A second implementation challenge involved enforcing governance rules outside the user interface. Hiding an approval button is not sufficient authorization. I therefore created workflow-level validation that only Reviewer or Administrator roles may record a final decision, prevents a requester from reviewing their own submission, restricts final decisions to Approved, Conditional, or Rejected, and requires a rationale. This makes the rule portable to a future server-side implementation. The main challenge was balancing MVP simplicity with meaningful safeguards. I resolved it by keeping the algorithm deterministic and small while adding explicit escalation and authorization rules instead of introducing machine learning or complex automation.

```

1 // Human decision safeguards
2 const REVIEW_ROLES = new Set(["Reviewer", "Administrator"]);
3 const FINAL_DECISIONS = new Set(["Approved", "Conditional", "Rejected"]);
4
5 export function canRecordDecision(role) {
6   return REVIEW_ROLES.has(role);
7 }
8
9 export function validateDecision({
10   role,
11   requesterId,
12   reviewerId,
13   decision,
14   rationale
15 }) {
16   if (!canRecordDecision(role)) {
17     return { ok: false, error: "Only a Reviewer or Administrator may record a decision." };
18   }
19
20   if (requesterId && reviewerId && requesterId === reviewerId) {
21     return { ok: false, error: "A requester cannot approve or reject their own request." };
22   }
23
24   if (!FINAL_DECISIONS.has(decision)) {
25     return { ok: false, error: "Decision must be Approved, Conditional, or Rejected." };
26   }
27
28   if (!String(rationale ?? "").trim()) {
29     return { ok: false, error: "A reviewer rationale is required for a final decision." };
30   }
31
32   return { ok: true };
33 }
34
35 export function createDecisionRecord(input, now = new Date()) {
36   const validation = validateDecision(input);
37   if (!validation.ok) {
38     throw new Error(validation.error);
39   }
40
41   return {
42     requestId: input.requestId,
43     decision: input.decision,
44     rationale: input.rationale.trim(),
45     reviewerId: input.reviewerId,
46     recordedAt: now.toISOString()
47   };
48 }
49
50 export function createAuditEvent({

```

Figure 3. Workflow safeguards enforcing reviewer authority, separation of duties, and rationale.

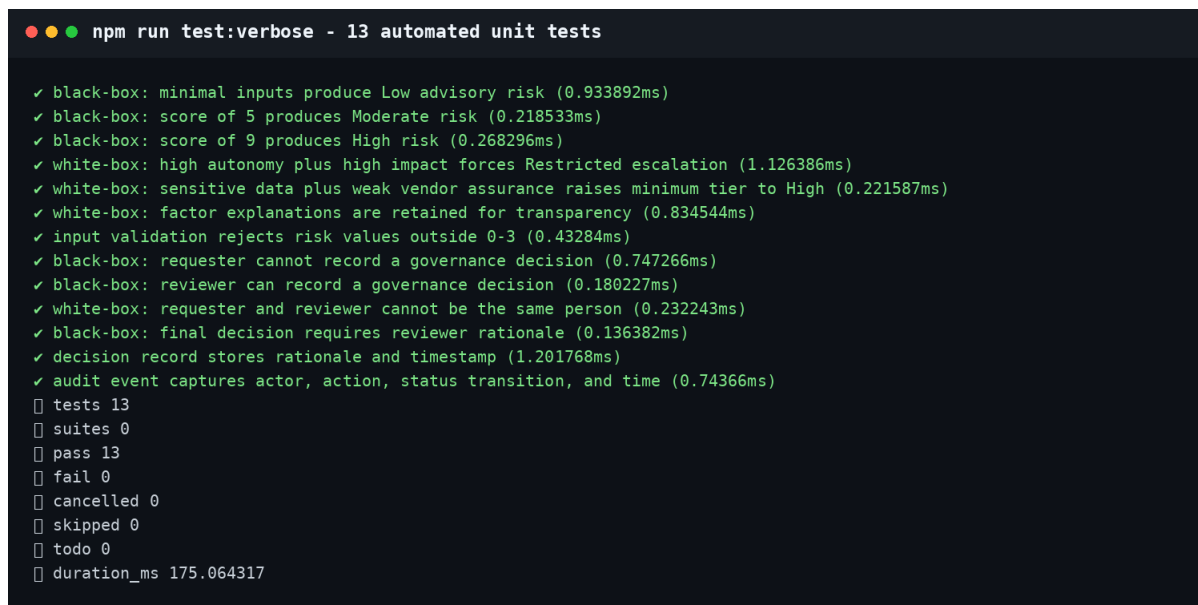
Part B: Unit Testing of Functional Modules

Testing Approach and Results

I used Node.js's built-in node:test framework with strict assertions. This is an appropriate JavaScript unit-testing framework and avoids adding unnecessary dependencies to the early MVP. I performed both black-box and white-box unit testing. Black-box tests validate observable behavior without relying on internal structure, such as verifying that a score of 5

returns Moderate risk or that a Requester cannot record a decision. White-box tests deliberately exercise known internal branches, including mandatory escalation rules and the separation-of-duties condition that prevents self-approval. Boundary and negative testing were also represented by checking threshold values and rejecting factor inputs outside the permitted 0-3 range.

The automated suite contains 13 tests. Seven focus on the risk engine: Low behavior, the Moderate and High boundaries, Restricted escalation, vendor-assurance escalation, retained explanations, and invalid input. Six focus on workflow and audit logic: Requester denial, Reviewer permission, self-review prevention, required rationale, decision record creation, and audit-event creation. The local execution and the GitHub Actions CI run both completed with 13 tests passed and zero failures. The same CI workflow also runs the repeatable core-logic demonstration, so a code change that breaks the algorithm can be detected before integration.



```
● ● ● npm run test:verbose - 13 automated unit tests

✓ black-box: minimal inputs produce Low advisory risk (0.933892ms)
✓ black-box: score of 5 produces Moderate risk (0.218533ms)
✓ black-box: score of 9 produces High risk (0.268296ms)
✓ white-box: high autonomy plus high impact forces Restricted escalation (1.126386ms)
✓ white-box: sensitive data plus weak vendor assurance raises minimum tier to High (0.221587ms)
✓ white-box: factor explanations are retained for transparency (0.834544ms)
✓ input validation rejects risk values outside 0-3 (0.43284ms)
✓ black-box: requester cannot record a governance decision (0.747266ms)
✓ black-box: reviewer can record a governance decision (0.180227ms)
✓ white-box: requester and reviewer cannot be the same person (0.232243ms)
✓ black-box: final decision requires reviewer rationale (0.136382ms)
✓ decision record stores rationale and timestamp (1.201768ms)
✓ audit event captures actor, action, status transition, and time (0.74366ms)
  tests 13
  suites 0
  pass 13
  fail 0
  cancelled 0
  skipped 0
  todo 0
  duration_ms 175.064317
```

Figure 4. Unit-test execution showing 13 tests passed and zero failures.

Testing identified several issues that were easy to miss in the visual prototype. First, UI-only role restrictions did not protect the underlying business rule, so authorization validation was moved into the workflow module. Second, reviewer rationale needed to be programmatically

required rather than treated as an optional form behavior. Third, threshold tests showed why score-only classification was insufficient for high-consequence combinations, which reinforced the mandatory escalation design. These changes demonstrate how unit tests improve reliability: defects are isolated close to the module where they originate, and each module can change without requiring the entire application to be retested manually. Black-box tests protect expected behavior, white-box tests cover important branches, boundary tests protect thresholds, negative tests verify rejection of invalid actions or inputs, and regression tests prevent previously corrected behavior from returning.

Part C: Version Control Management

Repository, Branching, Tags, and Releases

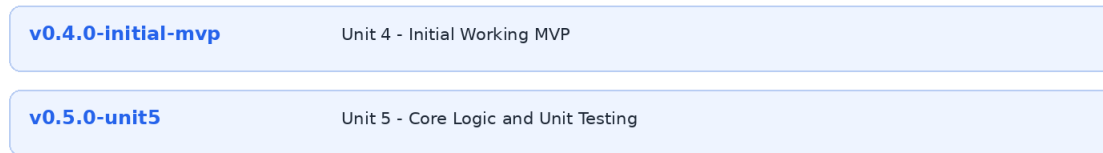
Public GitHub repository: <https://github.com/ctucker-web/ai-governance-capstone>

The repository uses a simple layered branching model. main represents the stable project baseline, development is the integration branch, and short-lived feature branches isolate significant work. The Unit 4 prototype was developed on feature/initial-mvp. Unit 5 was developed on feature/unit5-core-logic from the Unit 4 milestone so that algorithm and test changes remained separate and reviewable. Pull Request #3 contains the Unit 5 change set. This approach provides traceability from a requirement to a feature branch, commit, automated test run, pull request, tag, and release.

Version-control milestones



Tags / GitHub Releases



Public repository: github.com/ctucker-web/ai-governance-capstone

Figure 5. Branch and milestone structure used for the Unit 4 and Unit 5 work.

I created two milestone tags and corresponding GitHub releases. `v0.4.0-initial-mvp` records the first working browser MVP. `v0.5.0-unit5` records the implementation of modular core logic and automated unit testing. The releases were created through a GitHub Actions milestone workflow, demonstrating an advanced version-control practice in which repeatable automation manages release metadata instead of relying only on manual steps. The Unit 5 CI run also completed successfully after checking required documentation, executing the unit tests, and validating the example output.

Unit 5 Pull Request: [Pull Request #3](#)

Unit 4 release: [v0.4.0-initial-mvp](#)

Unit 5 release: [v0.5.0-unit5](#)

Successful CI run: [GitHub Actions run 10](#)

Tags provide fixed references to milestone commits, making it possible to reproduce or inspect the exact state submitted at a particular stage. Releases add human-readable milestone notes and downloadable source archives. Feature branches reduce the risk of destabilizing integration work, while pull requests create a visible review boundary. Together, branches, CI,

tags, and releases make progress easier to monitor and provide evidence that the capstone is being developed as a controlled software-engineering project rather than as a collection of untracked files.

Conclusion

Unit 5 strengthened the MVP by moving central governance rules into modular, tested code. The advisory risk engine remains explainable and subordinate to human judgment, while workflow validation protects reviewer accountability and auditability. Thirteen automated tests and a successful GitHub Actions run provide repeatable evidence that the modules behave as expected. The version-control milestones also establish a clear history from initial MVP through tested core logic, creating a stronger foundation for later database, authentication, integration, and evaluation work.